Gráfalgoritmusok 2

Horváth Gyula
horvath@inf.elte.hu

1. Topologikus rendezés

1.1. definíció. *Egy* `G = (V, E)` *irányított gráf topologikus rendezésén a* `V` *pontjainak egy olyan* $(v_1, v_2, \ldots, v_n)$ *(*`V = |V|`*) felsorolását értjük, amelyre teljesül:*
Minden `(u, v)` $\in$ `E` *élre,* `u` *előbb áll a felsorolásban, mint* `v`*.*

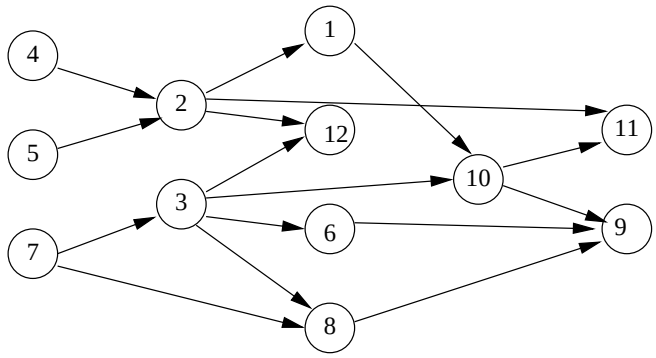
1.2. lemma. *A* `G = (V, E)` *irányított gráfnak akkor és csak akkor van topologikus rendezése, ha* `G` *körmentes.*

1.3. lemma. *A* `G = (V, E)` *irányított gráfban akkor és csak akkor van kör, ha van vissza-éle.*

1.4. lemma. *A mélységi bejárás során, amikor a* `p` *pont színét feketére állítjuk, tegyük a sorozat elejére* `p` *pontot. Az így kapott sorozat egy topologikus rendezés lesz.*

1.1. Bizonnyítás

Amikor a `Szin[p] = Fekete` utasítást végrehajtjuk, akkor már minden olyan pontot elértünk, és beraktuk a sorozatba, amelyek elérhetők `p`-ből.



1. ábra. Körmentes gráf

```
typedef
vector<int> Graf[maxN];
enum Paletta {Feher, Szurke, Fekete};
Graf G;
int T[maxN];
Paletta Szin[maxN];
int FeFa[maxN];
int u;//a T-sor utolsó tagja
```

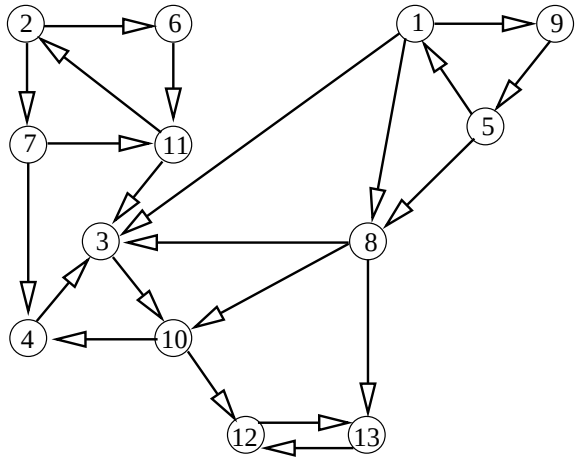
```
1  void MelyBejar(int p){
2  //Globális: G, T, u, Szin
3  //u kezdeti értéke n
4  Szin[p]=Szurke;
5  for (int q:G[p]){//p->q él
6    if (Szin[q]==Feher){//q-ban még nem jártunk
7      FeFa[q]=p;
8      MelyBejar(q);
9    }else if(Szin[q]==Szurke){//visszaél
10     return;
11   }
12 }
13 Szin[p]=Fekete;
14 T[u--]=p;
15 }
16 void Beolvas(Graf& G, int& n, int& m);
17 void Toprend(Graf& G){
18   //Globális: T
19   int n,m;
20   Beolvas(G, n, m);
21   u=n;
22   for(int p=1;p<=n;p++) Szin[p]=Feher;
23   for(int p=1;p<=n;p++)
24   if(Szin[p]==Feher)
25   MelyBejar(p);
26   if(u>0){
27     cout<<"A gráf nem körmentes!\n";
28   }else{
29     for(int i=1;i<=n;i++)
30     cout<<T[i]<<" "; cout<<endl;
31   }
32 }
```

2. Erősen összefüggő komponens fogalma

Tekintsük a $G = (V, E)$ irányított gráfot.

2.1. definíció. *Azt mondjuk, hogy a* G *gráf* p *és* q *pontja ekvivalens, jelben* $p \simeq q$ *ha van* p*-ből* q*-ba út:* $p \rightsquigarrow q$ *és van* q*-ból* p*-be is út:* $q \rightsquigarrow p$.

Nyilvánvaló, hogy a $\simeq$ reláció ekvivalencia reláció, tehát a V ponthalmaz egy osztályozását adja.



2. ábra. Példa gráf

2.2. definíció. *Jelölje* $[p]$ *a* p *pontot tartalmazó* $\Leftrightarrow$ *reláció szerinti ekvivalencia osztályt, azaz*

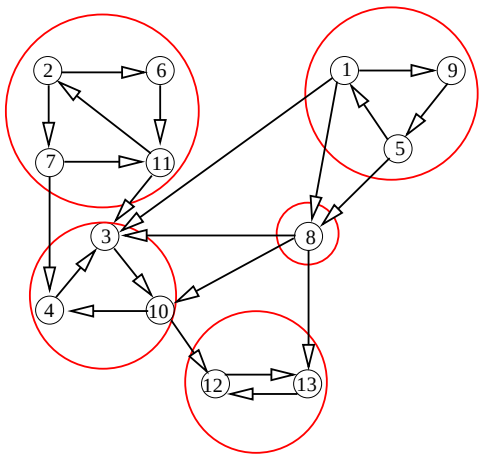
$$[p] = \{q \in V : p \simeq q\}$$

2.3. definíció (Komponens-gráf). *A* $G = (V, E)$ *irányított gráf komponens-gráfja az a* $G_\simeq = (V_\simeq, E_\simeq)$, *ahol a pontok az ekvivalencia osztályok:*

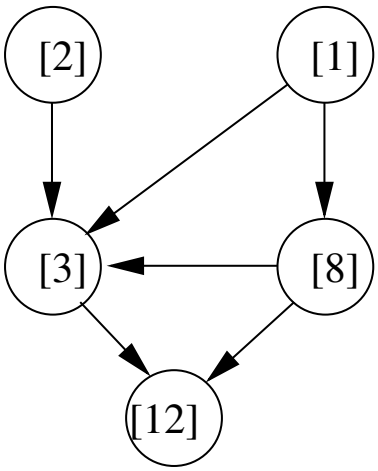
$$V_\simeq = \{[p] : p \in V\}$$

élei pedig: $[p] \to [q]$ *akkor és csak akkor, ha van olyan* $u \in [p]$ *és olyan* $v \in [q]$, *hogy* $u \to v \in E$

$$E_\simeq = \{([p], [q]) : \exists u \in [p] \text{ és } \exists v \in [q] \text{ hogy } (u, v) \in E\}$$



3. ábra. A példa gráf erősen összefüggő komponensei: $[2] = \{2, 6, 7, 11\}$, $[1] = \{1, 9, 5\}$, $[3] = \{3, 4, 10\}$, $[8] = \{8\}$, $[12] = \{12, 13\}$



4. ábra. A példa gráf komponens-gráfja: $V_\simeq = \{[2], [1], [8], [3], [12]\}$, $E_{[]} = \{[2] \to [3], [1] \to [8], [1] \to [3], [8] \to [3], [8] \to [12], [3] \to [12]\}$

2.4. definíció. *Minden* $p \in V$ *pont esetén a* $[p]$ *halmazt a gráf erősen összefüggő komponensének nevezzük.*
Ha a gráfnak egyetlen erősen összefüggő komponense van, azaz ha $V = [p]$ *valamely* $p \in V$*-re, akkor azt mondjuk, hogy a gráf erősen összefüggő.*

2.5. Állítás. *Bármely* G *gráf* $G_{\simeq}$ *komponens-gráfja körmentes.*

2.1. Bizonyítás

Ha lenne a komponens-gráfban kör, akkor a körben lévő komponensek elemei mind ekvivalensek lennének.

2.2. Ősapa fogalma

Tekintsünk a $G = (V, E)$ gráf egy tetszőleges mélységi bejárását. Azt mondjuk, hogy a $p \in V$ pontot ősapának nevezzük (a mélységi bejárásra nézve), ha $[p]$ komponensben az elsőként éri el a bejárás.

2.6. Állítás. *Legyen* $T = (v_1, v_2, \ldots, v_n)$ *az a sorozat, amelyet a topologikus rendezésnél előállított az algoritmus. Azaz, az elhagyási idő szerint fordított sorrend. Ekkor minden* p *és* q *ősapára teljesül, hogy ha van* $p \rightsquigarrow q$ *út, de nincs* $q \rightsquigarrow p$ *út, akkor* p *előbb van a* T *sorozatban, mint* q.

Az állítás következik abból, hogy a komponens-gráf körmentes. Az olyan T sorozatot, amire teljesül a fenti állítás feltétele, kvázi-topologikus sorrendnek nevezzük.

2.7. definíció. *A* $G = (V, E)$ *irányított gráf olyan* $D \subseteq V$ *legkisebb elemszámú ponthalmazát, amely teljesül:*

$$\textit{bármely } q \in V \textit{ ponthoz van olyan } p \in D, \textit{ hogy } p \rightsquigarrow q$$

dominátornak *nevezzük.*

2.8. definíció. *A* $G = (V, E)$ *irányított gráf olyan* $C \subseteq V$ *legkisebb elemszámú ponthalmazát, amely teljesül:*

$$\textit{bármely } q \in V \textit{ ponthoz van olyan } p \in C, \textit{ hogy } q \rightsquigarrow p$$

kodominátornak *nevezzük.*

2.3. Feladat: dominátor halmaz kiszámítása

Állítsuk elő a T sorozatot a MelyBejar1 eljárással.
A T sorozat első v_1 tagja nyilván ősapa, $[v_1]$ pedig 0-befokú a komponens-gráfban, tehát a dominátor halmaznak tartalmaznia kell $[v_1]$ valamely (tetszőleges elemét). Vegyük be v_1-et D-be, majd a MelyBejar2 eljárással fessük be Feketére a v_1-ből elérhető pontokat. Az után T-ben az első olyan pont, amely Szurke, szintén egy olyan komponens ősapja, amely 0-befokú a komponens-gráfban. Folytassuk a Feketére festést, amíg a T sorozatban van nem Fekete pont.

```
1 void MelyBejar1(int p){
2   //Globalis: G,T,Szin,u
3   Szin[p]=Szurke;
4   for (int q:G[p])
5   if (Szin[q]==Feher)
6   MelyBejar1(q);
7   T[u--]=p;
8 }
9 void MelyBejar2(int p){
```

```
  Szin[p]=Fekete;
  for (int q:G[p])
  if (Szin[q]==Szurke)
  MelyBejar2(q);
}

void Dominator( vector<int>& D){
  //Globális: u, n
  u=n;
  for(int x=1;x<=n;x++) Szin[x]=Feher;
  for(int x=1;x<=n;x++)
  if(Szin[x]==Feher)
  MelyBejar1(x);

  for(int i=1;i<=n;i++){
    int x=T[i];
    if(Szin[x]==Szurke){
      D.push_back(x);
      MelyBejar2(x);
    }
  }
}
```

Nyilvánvaló, hogy ha `D` dominátora a `G = (V, E)` gráfnak, akkor kodominátora a `Gt` transzponált gráfnak. Tehát a kodominátor kiszámítható úgy, mint a `Gt` transzponált gráf dominátora.

3. Erősen összefüggő komponensek kiszámítása

3.1. Kosaraju algoritmusa

Egy `G = (V, E)` irányított gráf transzponáltja az a `Gt = (Vt, Et)` gráf, ahol `V = Vt` és `(p, q)` akkor és csak elem a `Et` élhalmaznak, ha `(q, p)` $\in$ `E`. Nyilvánvaló, hogy a `Gt` transzponált gráfnak ugyanazok az erősen összefüggő komponensei. Legyen `R =` $(r_1, \ldots, r_k)$ az ősapák topologikus sorrendje, tehát az ősapák sorrendje a `T` sorozatban. Ha az ősapkra híjuk a MelyBejarT mélységi bejárást a `Gt` gráfban, akkor minden egyes hívás egy erősen összefüggő komponens jár be.

```
typedef
vector<int> Graf[maxN];
int n;//pontok száma
Graf G;
Graf Gt;//a transzponált gráf
enum Paletta {Feher, Szurke, Fekete};
```

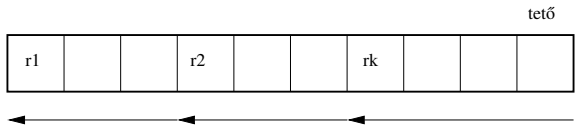
```
Paletta Szin[maxN];
int T[maxN];
int u;
vector<vector<int>> EOK;

void MelyBejar(int p){
  //Globális: G, T, u, Szin
  //u kezdeti értéke n
  Szin[p]=Szurke;
  for(int q:G[p]){//p->q él
    if (Szin[q]==Feher)//q-ban még nem jártunk
    MelyBejar(q);
  }
  T[u--]=p;
}

void MelyBejarT(int p, vector<int>& komp){
  //Global: Gt, Szin, EOK
  komp.push_back(p); //p bevétele a komponensbe
  Szin[p]=Fekete;
  for (int q:Gt[p])
  if (Szin[q]==Szurke)
  MelyBejarT(q, komp);
}
void EOKomp(Graf& G, vector<vector<int>>& EOK){
  u=n;
  for (int p=1; p<=n; p++) Szin[p]=Feher;
  for (int p=1; p<=n; p++)
  if (Szin[p]==Feher) MelyBejar(p);
  vector<int> komp;
  for(int x=1;x<=n;x++){
    int p=T[x];
    if(Szin[p]==Szurke){
      komp.clear();
      MelyBejarT(p,komp);
      EOK.push_back(komp);
    }
  }
  cout<<EOK.size()<<endl;
```

```
46    for(auto komp:EOK){
47      for(int x:komp) cout<<x<< " ";
48      cout<<endl;
49    }
50  }
```

3.2. Tarjan algoritmusa



5. ábra. A verem szerkezete.

Alapötlet: A mélységi bejárás során rakjuk verembe a pontokat elérésükkor. Minden komponensből az ősapa kerül be először.
Legyen r_k az az ősapa, amelynek először fejeződik be a bejárása, azaz amikor Fekete színt kap. Ekkor a veremben az r_k pontig lévő pontok halmaza megegyezik az r_k-t tartalmazó erősen összefüggő komponenssel. Vegyük ki a veremből ezeket a pontokat és fessük be Pirosra. Ezt végezzük minden ősapa esetén.
A kérdés tehát az, hogy hogyan detektálhatjuk azt, hogy egy pont ősapa?
Ha a $p \to q$ él vizsgálatakor q színe Piros, az azt jelenti, hogy q nem a p-t tartalmazó komponensben van.
Az ősapa az egyetlen olyan pont a bejárás során, amelyből nem lehet eljutni nem Piros pontokon keresztül nálánál kisebb elérési pontba. Számítsuk azt az `L` függvényt, amelynek értéke a p pontra az alábbi értékek minimuma:

- `D[p]`
- `D[q]` ha $p \to q$ visszaél (Szin[q]==Szurke)
- `L[q]` ha $p \to q$ faél (Szin[q]==Feher)
- `D[q]` ha q színem nem Piros

Ekkor teljesül, hogy q akkor és csak akkor ősapa, ha befejezésekor `L[p] = D[p]`.

```
1  typedef
2  vector<int> Graf[maxN];
3
4  enum Paletta {Feher, Szurke, Fekete, Piros};
5  Graf G;
6  int D[maxN]; //elérési idö
7  int L[maxN]; //
8  Paletta Szin[maxN];
9  stack<int> V;
10 int n, kn=0, ido=0;
```

```
vector<vector<int>> EOK;

void MelyBejar(int p){
  //Globális: G, D, L, Szin, V, EOK
  ido++;
  D[p]=ido; L[p]=ido;
  V.push(p);
  Szin[p]=Szurke;
  for(int q:G[p]){
    if(Szin[q]==Feher){
      MelyBejar(q);
      L[p]=min(L[p],L[q]);
    }else if(Szin[q]!=Piros)
    L[p]=min(L[p],D[q]);
  }
  Szin[p]=Fekete;
  if(D[p]==L[p]){
    int x;
    vector<int> komp;
    do{
      x=V.top(); V.pop();
      Szin[x]=Piros;
      komp.push_back(x);
    }while(x!=p);
    EOK.push_back(komp);
  }
}
void Beolvas(int& n){
  int m,x,y;
  cin>>n>>m;
  for(int i=0;i<m;i++){
    cin>>x>>y;
    G[x].push_back(y);
  }
}
void Tarjan(){
  for(int p=1;p<=n;p++){
    Szin[p]=Feher;
  }
```

```
  for(int p=1;p<=n;p++){
    if(Szin[p]==Feher){
      MelyBejar( p);
    }
  }
}
int main(){
  ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
  Beolvas(n);
  Tarjan();

  cout<<EOK.size()<<endl;
  for(auto komp:EOK){
    for(int x:komp)
    cout<<x<< " ";
    cout<<endl;
  }
}
```

3.3. Körben lévő pontok kiszámítása

Nyilvánvaló, hogy `p` akkor és csak akkor van körben, ha `[p]` komponens elemszáma nem egy. Tehát ha kiszámítjuk az erősen összefüggő komponenseket, akkor ebből megkaphatjuk a körben lévő pontokat.
A Tarján algoritmus egy hatékonyabb megoldást kínál. Ugyanis bármely `p` akkor és csak akkor nincs körben, ha a befejezésekor `D[p] == L[p]` és a verem tetején `p` van.

```
if(D[p]==L[p]){
  if(p==V.top()) Korben[p]=false;
  int x;
  do{
    x=V.top(); V.pop();
    Szin[x]=Piros;
  }while(x!=p);
}
```

A Tarjan algoritmus alkalmazásával egyetlen mélységi bejárással elő tudjuk állítani az alábbi objektumokat:

R: az ősapák halmaza

Rep: a reprezentány függvény: `Rep[p]` `p` ősapja

Dom: a dominátor halmaz

Cod: a kodominátor halmaz

```
typedef
  vector<int> Graf[maxN];

enum Paletta {Feher, Szurke, Fekete, Piros};
Graf G;
int D[maxN]; //elérési idö
int L[maxN]; //
Paletta Szin[maxN];
stack<int> V;
int n, kn=0, ido=0;
vector<int> R;
vector<int> Dom;
vector<int> Cod;

int Rep[maxN];
int Befok[maxN];
int Kifok[maxN];
```

```
void MelyBejar(int p){
  //Globális: G, D, L, Szin, V, EOK
  ido++;
  D[p]=ido; L[p]=ido;
  V.push(p);
  Szin[p]=Szurke;
  for(int q:G[p]){
    if(Szin[q]==Feher){
      MelyBejar(q);
      L[p]=min(L[p],L[q]);
    }else if(Szin[q]!=Piros)
    L[p]=min(L[p],D[q]);
  }
  Szin[p]=Fekete;
  if(D[p]==L[p]){
    int x;
    R.push_back(p);
    do{
      x=V.top(); V.pop();
      Szin[x]=Piros;
      Rep[x]=p;
    }while(x!=p);
  }
}
```

```
void Tarjan(){
  for(int p=1;p<=n;p++){
    Szin[p]=Feher;
    Befok[p]=0;
    Kifok[p]=0;
  }
  for(int p=1;p<=n;p++){
    if(Szin[p]==Feher){
      MelyBejar( p);
    }
  }
  for(int x=1;x<=n;x++){
    for(int y:G[x]){
      if(Rep[x]!=Rep[y]){
        Kifok[Rep[x]]++;
        Befok[Rep[y]]++;
      }
    }
  }
  for(int x:R){
    if(Befok[x]==0) Dom.push_back(x);
    if(Kifok[x]==0) Cod.push_back(x);
  }
}
```

4. Komponensgráf előállítása

Azonosítsuk az erősen összefüggő komponenseket az ősapjukkal. Számítsuk ki a `G = (V, E)` gráfra az ősapák `R` halmazát és a `Rep : V` $\to$ `R` reprezentáns függvényt. Célunk a `KG = (R, KE)` komponens gráf előállítása.
Első lépésben minden `(u, v)` $\in$ `E` élre vegyük be a `KE` élhalmazba a `(Rep[u], Rep[v])` párt. Ez multigráf lesz, ugyanaz az él sokszor szerepelhet. Ha az aktuális probléma megoldás során ez nem zavaró, akkor készen vagyunk.
Ha nem megengedett, hogy `KG` multigráf legyen, akkor purgálással szüntessük meg a többszörös éleket.
Az ősapákat és a Rep függvényt előállíthatjuk akár a Kosaraju, akár a Tarján algoritmus alapján.

```
1  typedef
2  vector<int> Graf[maxN];
3
4  enum Paletta {Feher, Szurke, Fekete, Piros};
5  Graf G;
6  int D[maxN];//elérési idö
7  int L[maxN];//
8  Paletta Szin[maxN];
9  stack<int> V;
10 int n, ido=0;
11 vector<int> R;//ősapák
12 int Rep[maxN];//reprezentáns függvény
13 Graf KG;        //a komponens-grf
14
15 void MelyBejar(int p){
16   //Globális: G, D, L, Szin, V, R, Rep
17   ido++;
18   D[p]=ido; L[p]=ido;
19   V.push(p);
20   Szin[p]=Szurke;
21   for(int q:G[p]){
22     if(Szin[q]==Feher){
23       MelyBejar(q);
24       L[p]=min(L[p],L[q]);
25     }else if(Szin[q]!=Piros)
26     L[p]=min(L[p],D[q]);
27   }
28   Szin[p]=Fekete;
29   if(D[p]==L[p]){
30     int x;
31     R.push_back(p);
32     do{
33       x=V.top(); V.pop();
34       Szin[x]=Piros;
35       Rep[x]=p;
36     }while(x!=p);
37   }
38 }
```

```
81  void Purgalas(Graf& G, vector<int> R){
82    //többszörös élek eltüntetése
83    int Van[n+1];//inkrementális bitvektor
84    for(int x=1;x<=n;x++) Van[x]=0;
85    int h=0;
86    for(int r:R){
87      h++;
88      int i=0;
89      while(i<G[r].size()){
90        if(Van[G[r][i]]<h){//első előfordulás
91          Van[G[r][i]]=h;
92          i++;
93        }else{//már volt, G[r][i] törlése
94          swap(G[r][i], G[r][G[r].size()-1]);
95          G[r].pop_back();
96        }
97      }
98    }
99  }
100 void KomponensGraf(Graf& KG, vector<int>& R){
101   for(int p=1;p<=n;p++){
102     Szin[p]=Feher;
103   }
104   for(int p=1;p<=n;p++){
105     if(Szin[p]==Feher){
106       MelyBejar( p);
107     }
108   }
109   for(int x=1;x<=n;x++){
110     for(int y:G[x]){
111       if(Rep[x]!=Rep[y]){
112         KG[Rep[x]].push_back(Rep[y]);
113       }
114     }
115   }
116   Purgalas(KG, R);
117 }
```

```
void Beolvas(int& n){
  int m,x,y;
  cin>>n>>m;
  for(int i=0;i<m;i++){
    cin>>x>>y;
    G[x].push_back(y);
  }
}

int main(){
  ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
  Beolvas(n);
  KomponensGraf(KG, R);
  for(int x:R){
    cout<<x<<"->";
    for(int y:KG[x]) cout<<y<<","; cout<<endl;
  }
}
```

5. Erősen összefüggővé tétel

Feladat (IOI'1996): Adjuk meg, hogy legkevesebb hány új élet kell hozzávenni a `G = (V, E)` irányított gráfhoz, hogy az erősen összefüggő legyen!
Legyen `Dom` a dominátora, `Com` pedig a kodomitátora a `G` gráfnak.

Állítás

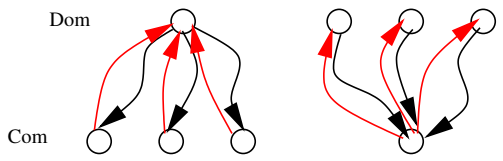
Legkevesebb $\max\{|Dom|, |Com|\}$ élet kell hozzávenni, hogy `G` erősen összefüggő legyen.

Állítás

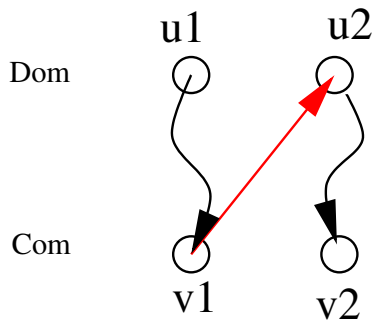
Minden $d \in Dom$ ponthoz van olyan $c \in Com$, hogy van $d \rightsquigarrow c$ út.
Minden $c \in Com$ ponthoz van olyan $d \in Dom$, hogy van $d \rightsquigarrow c$ út.

Bizonyítás

Bizonyítás `Dom` és `Com` elemszáma szerinti indukcióval.
Ha `Dom` vagy `Com` egyelemű, akkor az állítás nyilvánvalóan igaz. Tegyük fel, hogy $\max\{|Dom| > 1$ és $\max\{|Com| > 1$ Ekkor biztosan van olyan $u_1 \neq u_2 \in Dom$ és olyan $v_1 \neq v_2 \in Dom$, hogy van $u_1 \rightsquigarrow v_1$ és $u_2 \rightsquigarrow v_2$ út. Vegyük fel a $v_1 \to u_2$ élet. Ekkor eggyel csökken a gráfban a dominátorok és a kodominátorok száma.



6. ábra



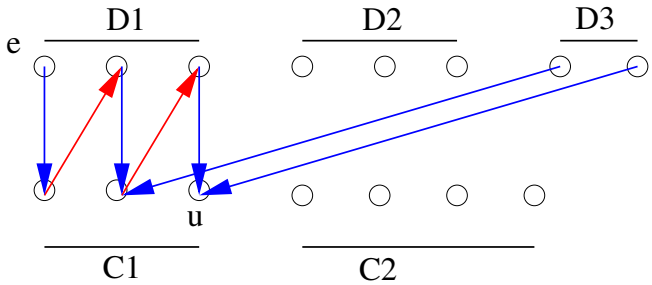
7. ábra

A fenti bizonyítás alapján kifejlesztett algoritmus nem lenne hatékon az új élek meghatározására, mert az élek számával arányos megoldást akarunk. Számítsunk egy olyan $\mathtt{D2C} : \mathtt{Dom} \to \mathtt{Com}$ függvényt, amelyre teljesül, hogy bármely $\mathtt{d} \in \mathtt{Dom}$ van $\mathtt{d} \rightsquigarrow \mathtt{D2C(d)}$ út (G-ben, de ez ekvivalens azzal, hogy a komponens gráfban).
Számítsunk egy olyan $\mathtt{C2D} : \mathtt{Com} \to \mathtt{Dom}$ függvényt, amelyre teljesül, hogy bármely $\mathtt{c} \in \mathtt{Com}$ van $\mathtt{C2D(c)} \rightsquigarrow \mathtt{c}$ út (G-ben, de ez ekvivalens azzal, hogy a komponens gráfban).

Lépésről-lépésre meghatározunk egy új élet. Van két kitüntetett pont, $\mathtt{e}$ és $\mathtt{u}$. Minden lépes megtétele után teljesül az alábbi feltétel:

$$(\forall \mathtt{x} \in \mathtt{D1} \cup \mathtt{C1})\ (\exists \mathtt{e} \rightsquigarrow \mathtt{x} \text{ és } \exists \mathtt{x} \rightsquigarrow \mathtt{u})$$

Az alábbi halmazokkal dolgozunk, ahol mindig teljesül, hogy $\mathtt{Dom} = \mathtt{D1} \cup \mathtt{D2} \cup \mathtt{D3}$ és $\mathtt{Com} = \mathtt{C1} \cup \mathtt{C2}$



8. ábra. Kék él: $\mathtt{d} \to \mathtt{D2C(d)}$ kapcsolat, piros él: új él

D1 a már párosított Dom-beli pontok,

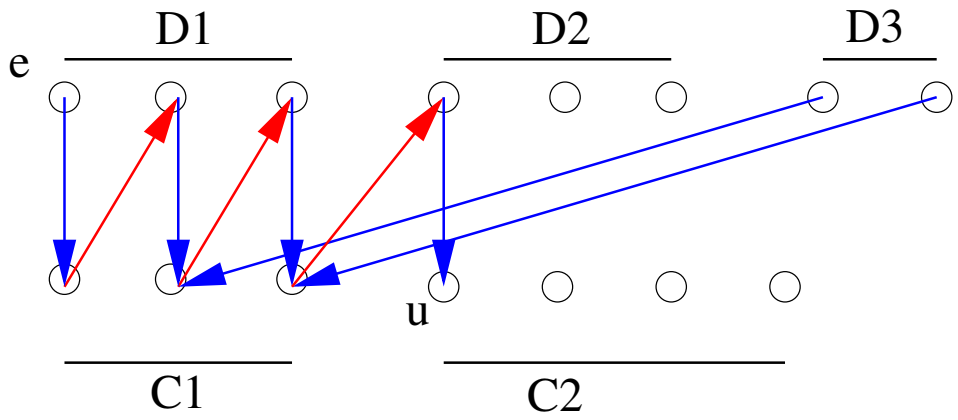
D2 a még nem vizsgált Dom-beli pontok,

D3 a már vizsgált Dom-beli pontok, de $\mathtt{D2C(d)} \in \mathtt{D1}$,

C1 a már párosított Com-beli pontok,

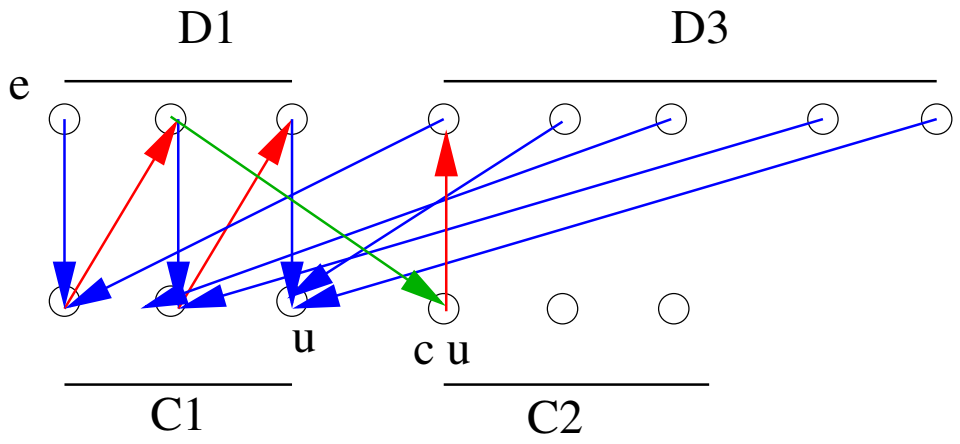
C2 a még nem párosított Com-beli pontok.

1. lépés: mindaddig, amíg `D2` nem üres, megyünk egy tetszőleges `d` elemét. Ha `D2C(d)` $\notin$ `C1`, akkor új él lesz `(u, d)` és `u = D2C(d)`, egyébként `d` átkerül a `D3` halmazba.

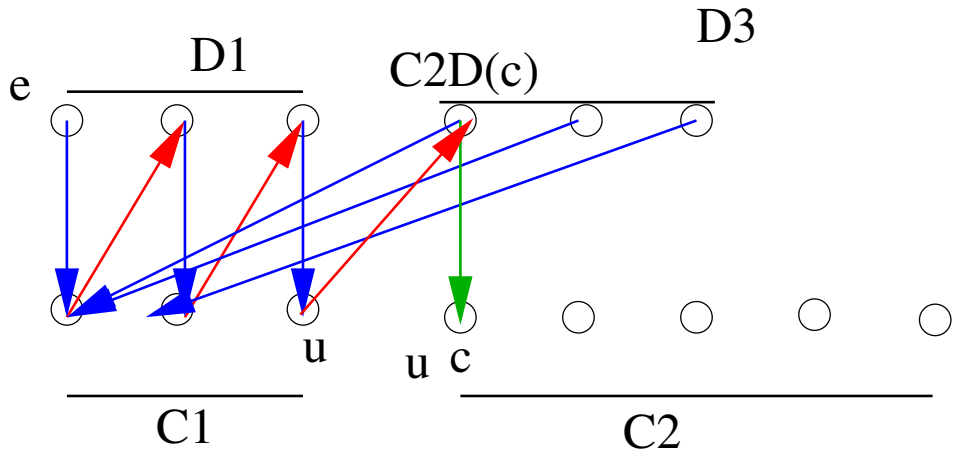


9. ábra. Kék él: `d` $\rightarrow$ `D2C(d)` kapcsolat, piros él: új él

2. lépés: mindaddig, amíg sem `C2`, sem `D3` nem üres, vegyük `C2` egy tetszőleges `c` elemét. Ha `C2D` $\in$ `D1`, akkor `(c, d)` új él lesz, ahol `d` a `D3` tetszőleges elem, ami átkerül `D1`-be és `u = c`. Egyébként, azaz ha `C2D)c)` $\in$ `D3`,akkor `(u, C2D(c))` lesz új él és `u = c`

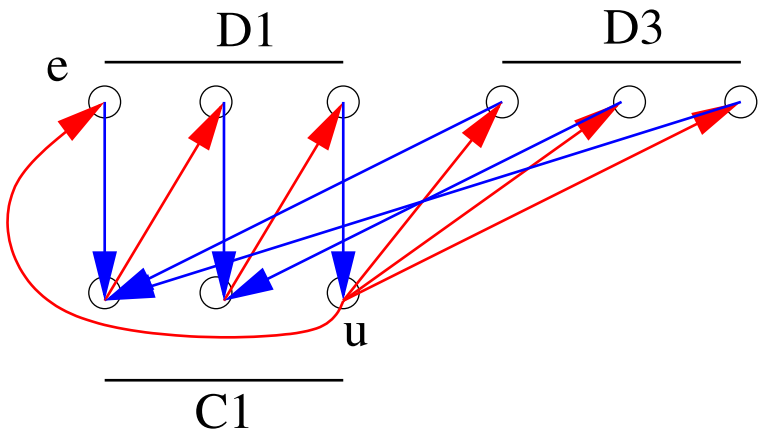


10. ábra. `C2D(c)` $\in$ `D1`, zöld él: `C2D(c)` $\rightarrow$ `c`



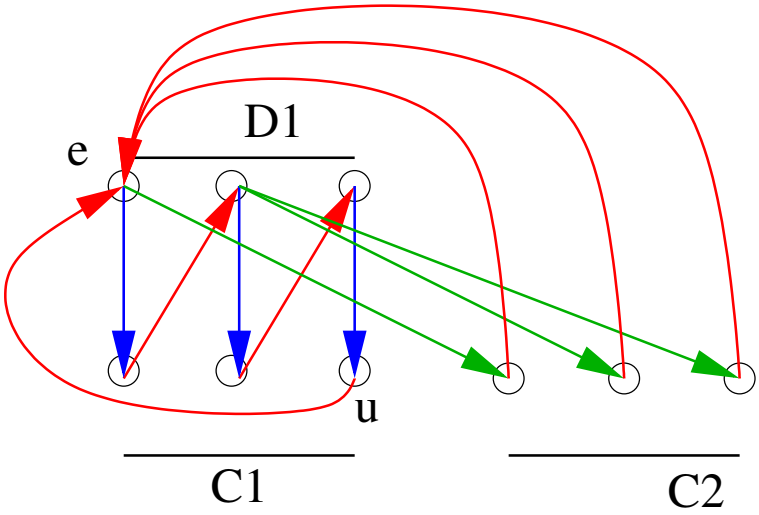
11. ábra. `C2D(c)` $\notin$ `D1`, piros él: új él, zöld él: `C2D(c)` $\rightarrow$ `c`, Kék él: `d` $\rightarrow$ `D2C(d)` kapcsolat

3.lépés:



12. ábra. `C2` = $\emptyset$, piros él: új él, Kék él: `d` $\rightarrow$ `D2C(d)` kapcsolat

4. lépés:



13. ábra. `D3` = $\emptyset$, piros él: új él, zöld él: `C2D(c)` $\rightarrow$ `c`, Kék él: `d` $\rightarrow$ `D2C(d)` kapcsolat

Hatékonyság

Mivel minden lépés futási ideje arányos az élek számával, így a teljes algoritmus is.

Megvalósítás

```
1  typedef vector<int> Graf[maxN];
2  enum Paletta {Feher, Szurke, Fekete, Piros};
3  Graf G;
4  int D[maxN];//elérési idö
5  int L[maxN];//
6  Paletta Szin[maxN];
7  stack<int> V;
8  int n, ido=0;
9  vector<int> R;//ősapák
10 int Rep[maxN];//reprezentáns függvény
11 Graf KG;        //a komponens-grf
12 Graf KGt; //komponens transzponáltja
13 vector<int> Dom;
14 vector<int> Cod;
15 int D2C[maxN];
16 int C2D[maxN];
17 bool D1[maxN];
18 bool C1[maxN];
19 vector<int> D3;
```

```
void MelyBejar(int p){
  //Tarjan algoritmus
  //Globális: G, D, L, Szin, V, R, Rep
  ido++;
  D[p]=ido; L[p]=ido;
  V.push(p);
  Szin[p]=Szurke;
  for(int q:G[p]){
    if(Szin[q]==Feher){
      MelyBejar(q);
      L[p]=min(L[p],L[q]);
    }else if(Szin[q]!=Piros)
    L[p]=min(L[p],D[q]);
  }
  Szin[p]=Fekete;
  if(D[p]==L[p]){
    int x;
    R.push_back(p);
    do{
      x=V.top(); V.pop();
      Szin[x]=Piros;
      Rep[x]=p;
    }while(x!=p);
  }
}
```

```
161 void Purgalas(Graf& G, vector<int> R){
162   //többszörös élek eltüntetése
163   int Van[n+1];//inkrementális bitvektor
164   for(int x=1;x<=n;x++) Van[x]=0;
165   int h=0;
166   for(int r:R){
167     h++;
168     int i=0;
169     while(i<G[r].size()){
170       if(Van[G[r][i]]<h){//első előfordulás
171         Van[G[r][i]]=h;
172         i++;
173       }else{//már volt, G[r][i] törlése
174         swap(G[r][i], G[r][G[r].size()-1]);
175         G[r].pop_back();
176       }
177     }
178   }
179 }
180 void KomponensGraf(Graf& KG, vector<int>& R){
181   for(int p=1;p<=n;p++){
182     Szin[p]=Feher;
183   }
184   for(int p=1;p<=n;p++){
185     if(Szin[p]==Feher){
186       MelyBejar( p);
187     }
188   }
189   for(int x=1;x<=n;x++){
190     for(int y:G[x]){
191       if(Rep[x]!=Rep[y]){
192         KG[Rep[x]].push_back(Rep[y]);
193       }
194     }
195   }
196   Purgalas(KG, R);
197 }
```

```
int Bejar(Graf& G, int DC[], int x){
  //D2C és C2D függvény készítése
  if(DC[x]>0) return DC[x];
  if(G[x].size()==0){
    DC[x]=x;
     return x;
  }
  for(int y:G[x])
  if(DC[y]==0){
     int d2cy=Bejar(G, DC, y);
    DC[y]=d2cy;
    DC[x]=DC[y];
     return d2cy;
  }else{
    DC[x]=DC[y]; return DC[y];
  }
}

void D2Ckeszit(Graf& G, vector<int> D, int DC[] ){
  for(int x:R) DC[x]=0;
  for(int x:D) if(DC[x]==0){
    DC[x]=Bejar(G, DC, x);
  }
}

void Beolvas(int& n){
  int m,x,y;
  cin>>n>>m;
  for(int i=0;i<m;i++){
     cin>>x>>y;
    G[x].push_back(y);
  }
}
```

```
int main(){
  ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);
  Beolvas(n);
  KomponensGraf(KG, R);
  if(R.size()==1){
    cout<<0<<endl;
    return 0;
  }
  for(int x:R){//Dom előállítása
    if(KG[x].size()==0) Cod.push_back(x);
    for(int y:KG[x])
    KGt[y].push_back(x);
  }
  for(int x:R){//Cod előállítása
    if(KGt[x].size()==0) Dom.push_back(x);
    D2C[x]=0; C2D[x]=0;
  }
  D2Ckeszit(KG, Dom, D2C);
  D2Ckeszit(KGt, Cod, C2D);
  for(int x:R){D1[x]=false; C1[x]=false;}
  vector<pair<int, int>> Ujelek;
  int e=Dom.back(); Dom.pop_back(); D1[e]=true;
  int u=D2C[e]; C1[u]=true;
```

```
    //1. lépés
    for(int d:Dom){
      int c=D2C[d];
      if(!C1[c]){
        Ujelek.push_back({u,d});
        D1[d]=true; C1[c]=true;
        u=c;
      }else{
        D3.push_back(d);
      }
    }
    //2. lépés
    for(int c:Cod){
      if(C1[c]) continue;
      while(D3.size()>0 && D1[D3.back()]) D3.pop_back();
      if(D3.size()==0){//4. lépés
        Ujelek.push_back({c,e});
        continue;
      }
      int d=C2D[c];
      if(D1[d]){
        d=D3.back(); D3.pop_back();
        Ujelek.push_back({c,d});
      }else{
        Ujelek.push_back({u,d});
      }
      D1[d]=true; u=c;
    }
    //3. lépés: C1=Cod
    for(int d:D3) if(!D1[d])
    Ujelek.push_back({u,d});

    Ujelek.push_back({u,e});
    cout<<Ujelek.size()<<endl;
    for(auto xy:Ujelek)
    cout<<xy.first<<" "<<xy.second<<endl;
}
```

6. Feladat 1:Optimális hálózat, https:/mester.inf.elte.hu/ Mélységi/87.

Az Onet kommunikációs hálózat terve N csomópontot tartalmaz. M számú csomópontpárt egyirányú kommunikációt biztosító közvetlen vonallal kötnek össze. A terv szerint a hálózat olyan, hogy bármely u csomópontból bármely másik v csomópontba pontosan egy útvonalon lehet adatot továbbítani. A terv kivitelezésébe azonban hiba csúszott, és pontosan egy közvetlen vonalat nem építettek meg. Készíts programot, amely meghatározza azt a két csomópontot, amelyek közötti közvetlen vonal hiányzik!

Bemenet

Bemenet A standard bemenet első sorában van a csomópontok N száma és a közvetlen vonalak M száma. A csomópontokat 1-től N-ig, a sorszámukkal azonosítjuk. A következő M sor mindegyike két egész számot tartalmaz, u_i és v_i, ami azt jelenti, hogy az u_i és v_i ($u_i \neq v_i$) között van kiépített közvetlen vonal, amelyen u_i-ből v_i-be lehet adatot továbbítani.

Kimenet

Kimenet A standard kimenet első sorába azt az M számot kell írni, ami a minimálisan szükséges mozgatások száma, amellyel a ládák sorba rakhatóak! A következő M sor egy lehetséges minimális mozgatási sorrendet adjon meg: a sorok mindegyike egy számpárt tartalmazzon egy szóközzel elválasztva! Minden i j számpár az i. helyen lévő láda mozgatását jelenti a j-edik helyre.

Példa

Bemenet

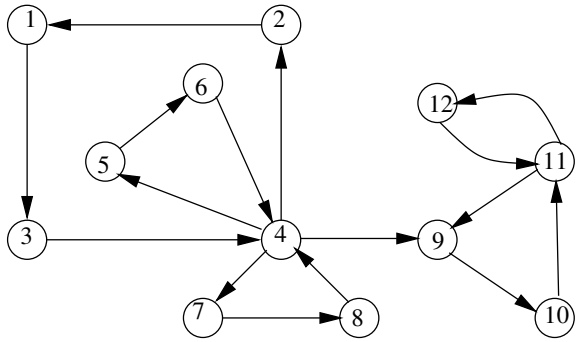
```
12 16
1 2
2 4
4 3
3 1
4 5
5 6
6 4
4 9
4 7
7 8
8 4
9 10
10 11
11 9
11 12
12 11
```

Kimenet

```
9 4
```

Korlátok

Korlátok $3 \leq N \leq 100\,000, N - 1 \leq M \leq 200\,000$
Időlimit: 0.2 másodperc
Memórialimit: 64 MB.



14. ábra

7. Feladat 2:Táblás játék, https:/mester.inf.elte.hu/ Mélységi/88.

Tekintsük azt a játékot, amelyet olyan pályán játszanak, amelyen `N` csomópont van. Bizonyos csomó-pont párokat egyirányú lépést lehetővé tevő közvetlen vonal köt össze. Minden csomó-pontban adott értékű kincs van. A játékos egy lépésben az tartózkodási helyéről valamelyik szomszédos csomópontba léphet. Bármely csomópontból indulhat, bármelyikben befejezheti a játékot. Bármely csomópontban akárhányszor járhat, de csak akkor kapja meg a csomó-pontban lévő kincset, amikor oda először lép. Készíts programot, amely kiszámítja azt a legnagyobb kincs összeget, amelyet a játékos szerezhet!

Bemenet

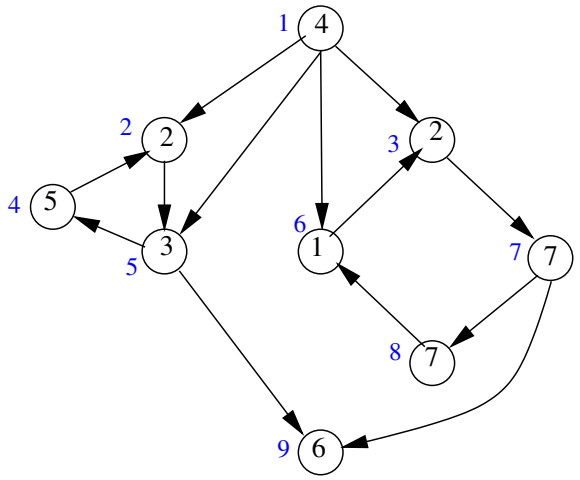
Bemenet A standard bemenet első sorában a csomópontok száma ($1 < N \leq 100\,00$) van.A második sorban pontosan N darab egész szám van, az i-edik szám K_i ($1 \leq K_i \leq 10\,000$) az i-edik csomó-pontban lévő kincs értéke. A csomó-pontokat az `1, ..., N` számokkal azonosítjuk. A következő N sor a csomó-pontok szomszédait adja meg. Az i-edik sorban azok a csomó-pontok vannak, amelyekbe egy lépéssel lehet lépni az i-edik csomó-pontból. A sorban az első szám a sorban a csomó-pontok száma.

Kimenet

Kimenet A standard kimenet első sorába a játék soron elérhető legnagyobb kincs értéket kell írni!

Példa

Bemenet	Kimenet
9 4 2 2 5 3 1 7 7 6 4 2 6 3 5 1 5 1 7 1 2 2 4 9 1 3 2 8 9 1 6 0	27



15. ábra. A körben a kincs értéke van, kivül a csomó-pont sirszáma.

Korlátok

Korlátok **Időlimit**: 0 . 2 másodperc
Memórialimit: 32 MB.